

INVOICE AND BILLING MANAGEMENT PLATFORM

RDK BILLS

EE4207 WEB APPLICATION DEVELOPMENT
MINI PROJECT

Department of Electrical and Information Engineering,
Faculty of Engineering
University of Ruhuna
Sri Lanka

NAME : ROSHAN.K
REG NO : EG/2023/5846
SEMESTER : 04

EG_2023_5846_bills

RDK Bills – Invoice and billing Management System

INTRODUCTION

RDK Bills is a web application developed to make invoice and client management easier. It allows users to create and view invoices, manage client information, check payment status, and search for invoice records.

The application has a frontend built using HTML, CSS and JavaScript. The backend uses Node.js and Express to process requests, while MongoDB stores the application data.

MAIN FEATURES

Invoice Management

The invoice management section allows users to create invoices by entering client details, invoice dates, item descriptions, quantities and prices. The system calculates line-item totals and the overall invoice total when creating an invoice.

Users can also view invoices, search by invoice number or client name, filter invoices according to payment status, and sort the invoice list. Pending invoices can be marked as paid, and unwanted invoices can be deleted.

The backend also provides an API endpoint for updating existing invoices.

Client Management

The client management section allows users to add and view client records. Each record can contain a client's name, email address, phone number and address.

Users can search for clients, update their information and delete records when necessary.

Validation and Error Handling

The application includes basic validation to check required information before saving records. Invoice items are checked for descriptions, quantities and prices.

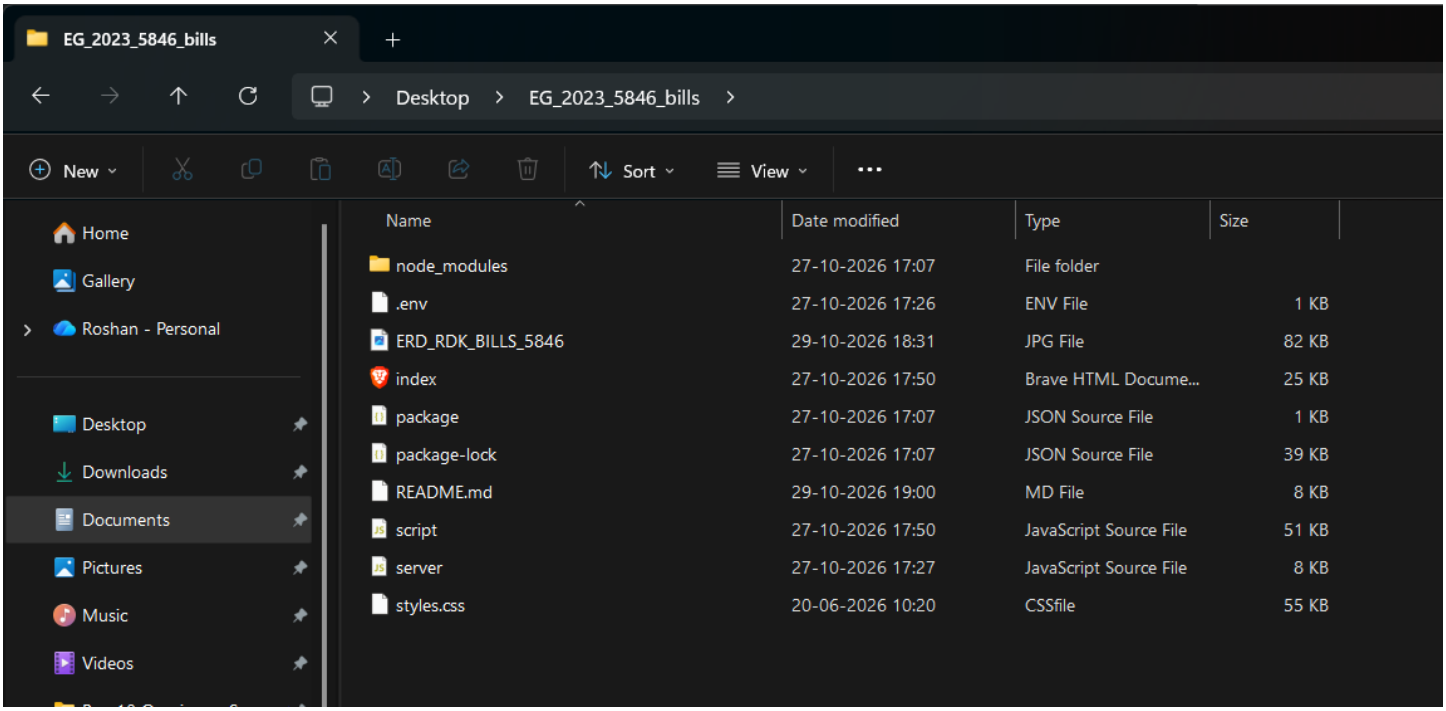
The backend also returns appropriate HTTP status codes and error messages when requests fail. Duplicate record IDs are handled by returning a conflict response.

TECHNOLOGIES USED

- HTML5 – Structures the web pages.
- CSS3 – Controls the appearance and layout of the application.
- JavaScript – Handles user interactions and communicates with the backend.
- Node.js – Runs the backend application.
- Express.js – Provides the API routes.
- MongoDB – Stores invoices and client records.
- Mongoose – Defines the database schemas and manages database operations.

PROJECT STRUCTURE

The main project files are organized as follows:



The index.html file contains the main web interface, while styles.css handles its design and layout. The script.js file manages frontend functionality and API requests.

The server.js file contains the backend routes and database models. The package files manage the Node.js dependencies, and the README provides information about the project and how to run it.

The .env file contains local environment settings and should not be uploaded with real credentials to a public repository. The .gitignore file should exclude sensitive files and generated folders such as node_modules.

REQUIREMENTS

To run the application locally, you need:

- Node.js and npm
- A MongoDB database, either locally or through MongoDB Atlas
- A modern web browser
- A code editor such as Visual Studio Code

HOW TO RUN THE PROJECT

- Step 1: Open the project

  Download or clone the project repository and open the project folder in a terminal.

  cd EG_2023_5846_bills

- Step 2: Install the dependencies

  Run the following command to install the required packages:

  npm install

- Step 3: Configure MongoDB

Create a .env file in the project root and add your MongoDB connection string and port.

MONGODB_URI=your_mongodb_connection_string
PORT=3000

- Step 4: Start the backend

Run: **node server.js**

The backend connects to MongoDB and starts the API server on port 3000 by default.

You can check whether the API is running by opening the following address in your browser:

http://localhost:3000/api/health

The expected response is:

{

"message": "RDK Bills API is running"

}

- Step 5: Open the frontend

Open the project in Visual Studio Code and use the Live Server extension to run index.html.

The frontend currently uses the following API address:

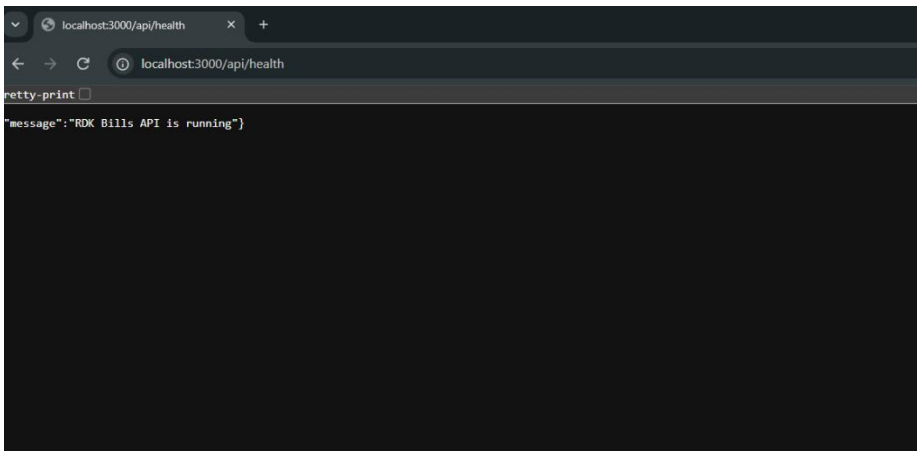
const API_BASE = 'http://localhost:3000/api';

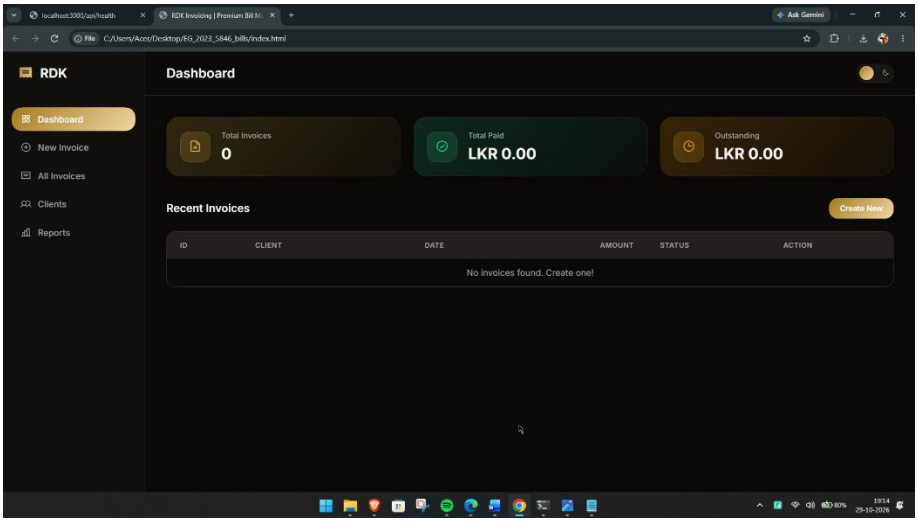
This address is suitable for local development when the backend is running on port 3000. If the backend is hosted elsewhere, the API address must be changed accordingly.

BACKEND API ENDPOINTS

The backend provides the following endpoints for working with invoices and clients.Health Check

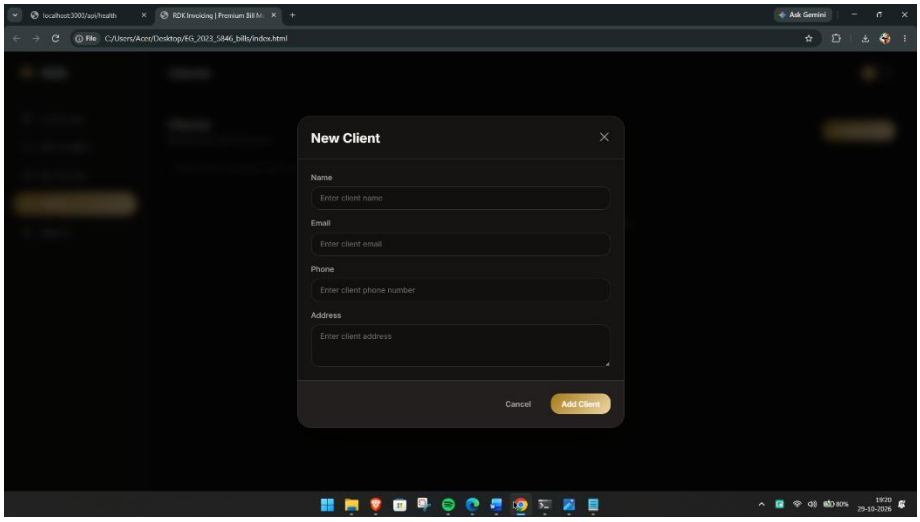
- GET /api/health Checks whether the API is running

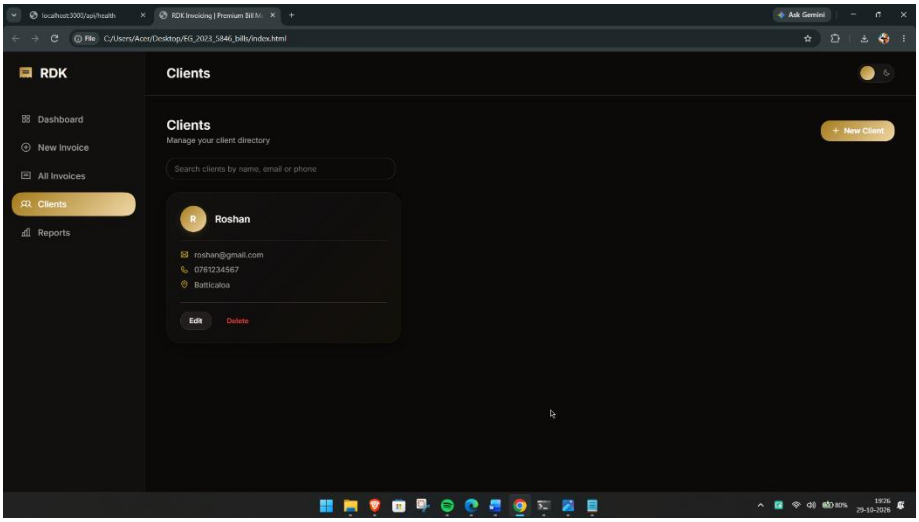




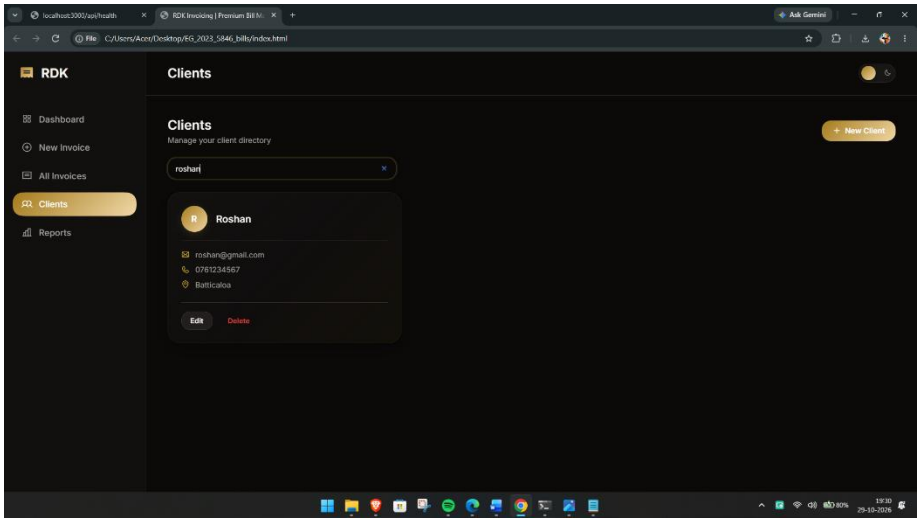
Client Endpoints

- POST /api/clients Adds a new client

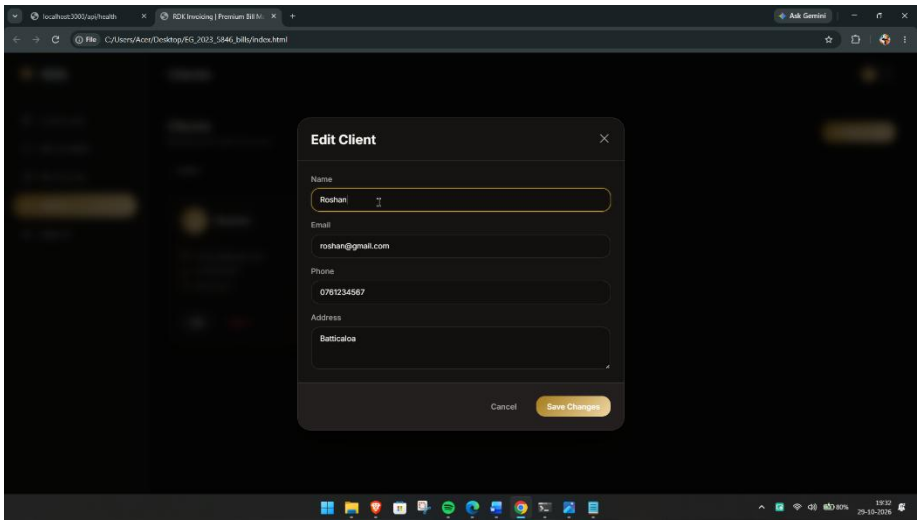




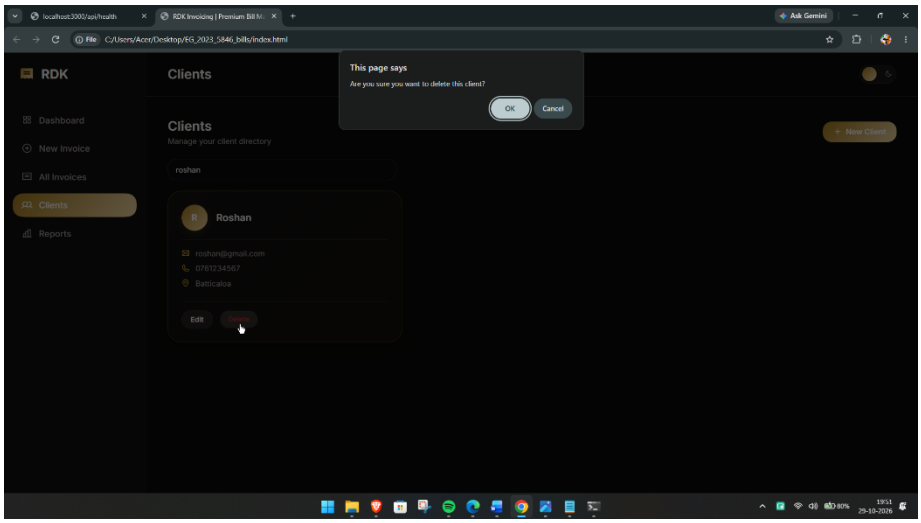
- GET /api/clients Retrieves client records



- PUT /api/clients/:id Updates client information

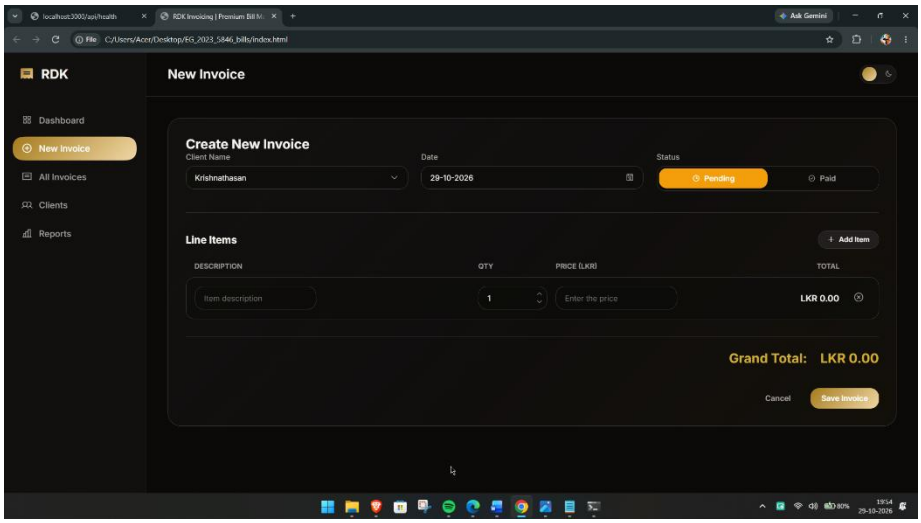


- DELETE /api/clients/:id Deletes a client

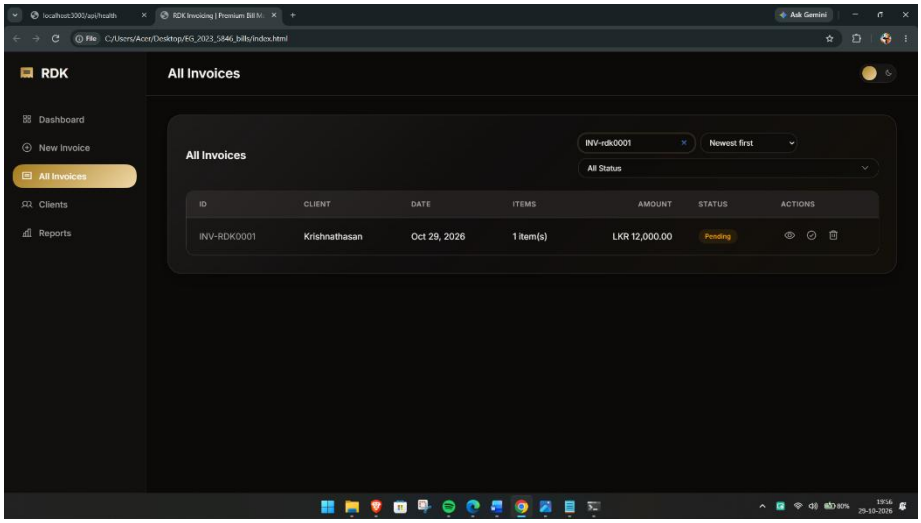


Invoice Endpoints

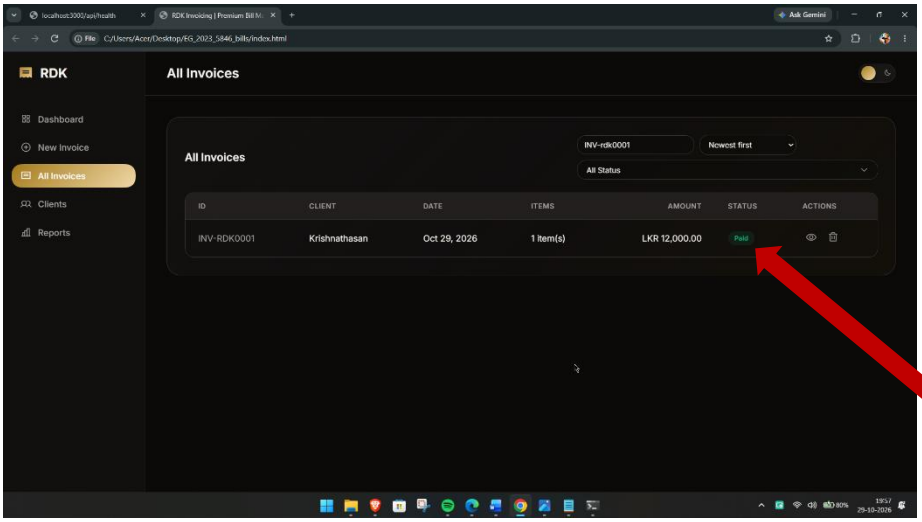
- POST /api/invoices Creates a new invoice



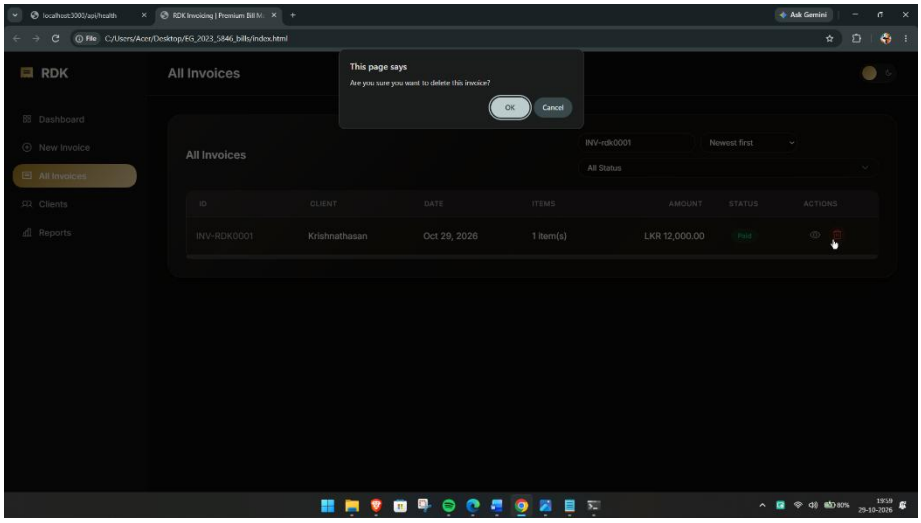
- GET /api/invoices Retrieves invoice records



- PUT /api/invoices/:id Updates an invoice



- DELETE /api/invoices/:id Deletes an invoice



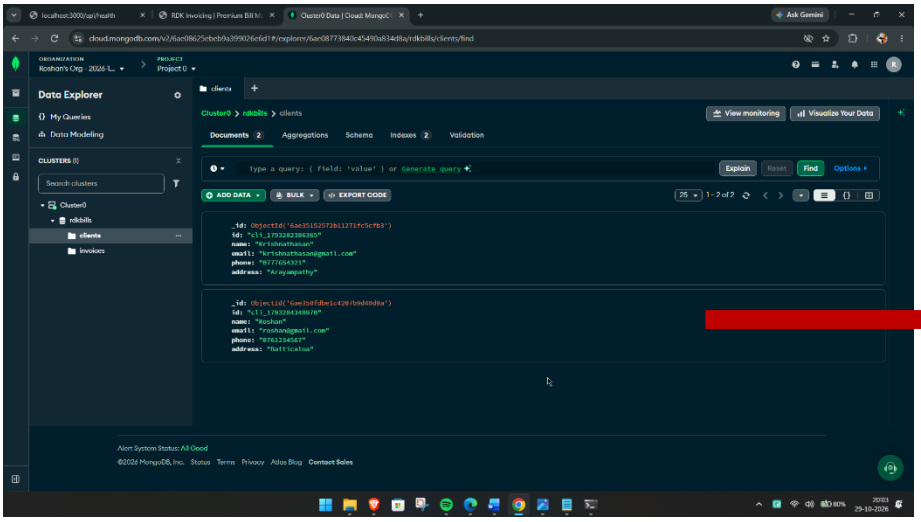
DATABASE DESIGN

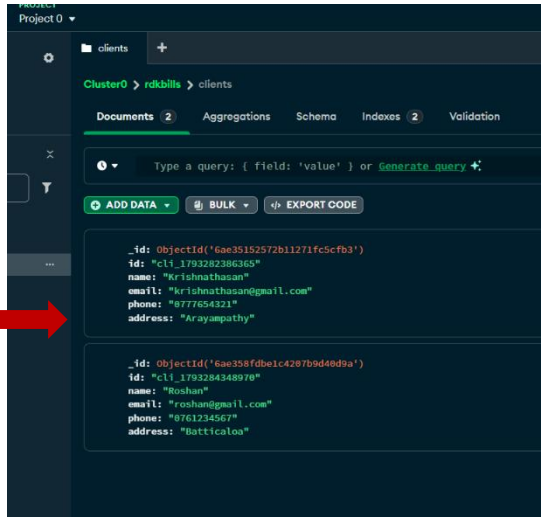
The application uses MongoDB to store client and invoice information in separate collections.

Client Collection

Each client record contains the following information:

- id – Unique client identifier
- name – Client's name
- email – Client's email address
- phone – Client's phone number
- address – Client's address



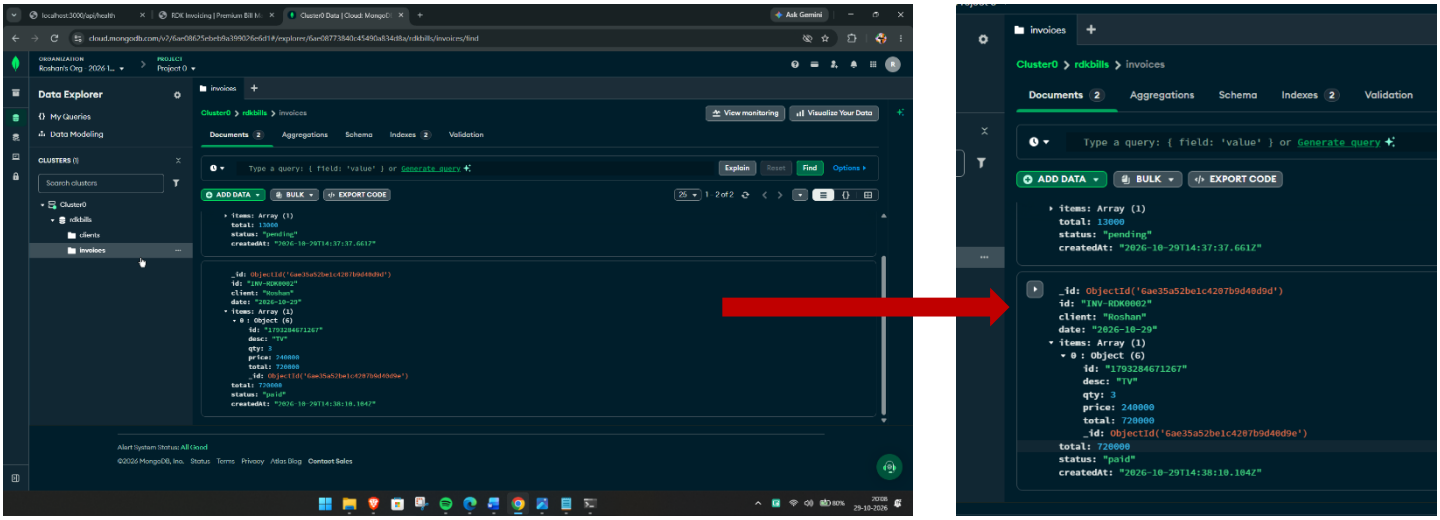


Invoice Collection

Each invoice record contains:

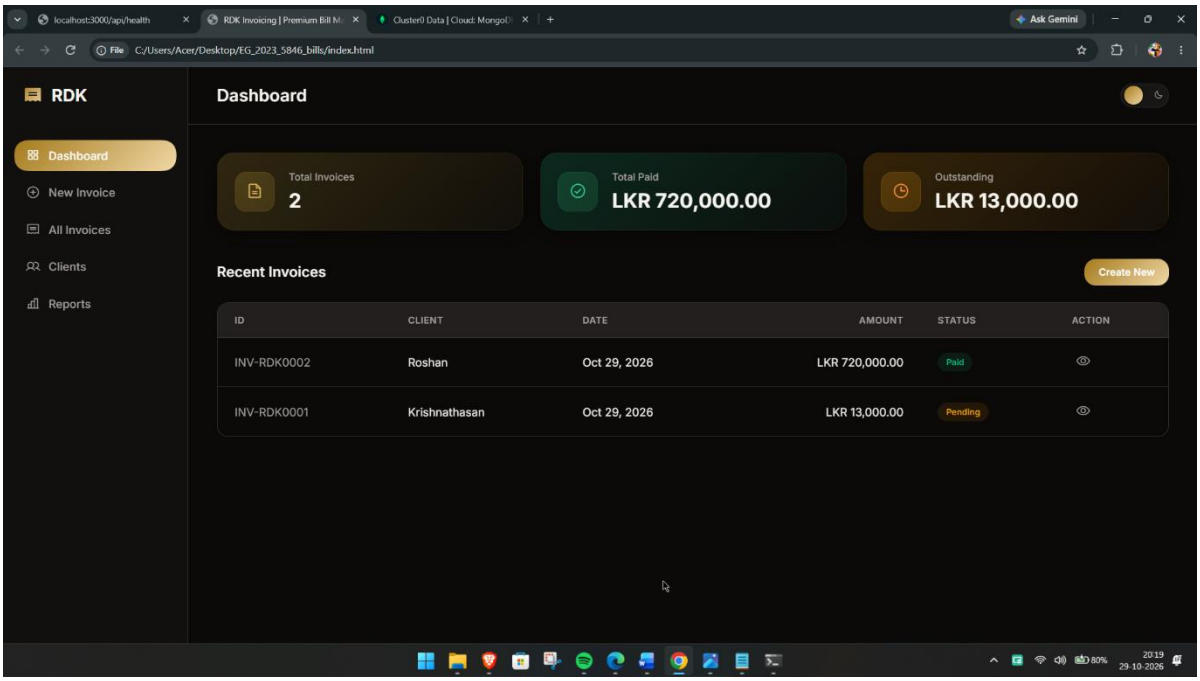
- id – Unique invoice identifier
- client – Client name stored in the invoice

- date – Invoice date
- items – An array containing the invoice items
- items[].id – Item identifier
- items[].desc – Item description
- items[].qty – Item quantity
- items[].price – Price per item
- items[].total – Total for the individual item
- total – Overall invoice amount
- status – Payment status, such as paid or pending
- createdAt – Invoice creation timestamp



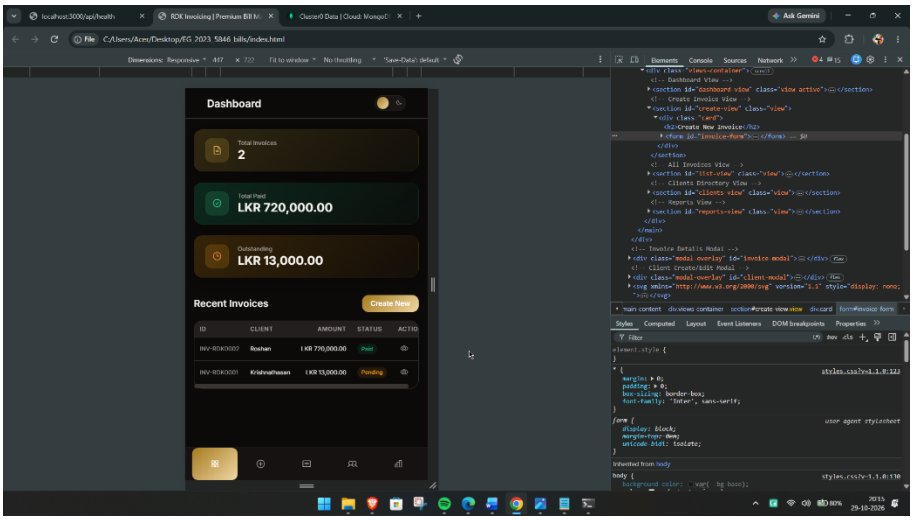
The invoice items are stored as embedded objects inside each invoice document rather than in a separate collection.

The client name is stored as text in the invoice. Therefore, the current implementation does not use a direct foreign-key relationship between the client and invoice collections.



TESTED

- Create a new invoice and confirm that it appears in the invoice list.
- Refresh the page and check whether the invoice remains saved.
- Search for invoices using an invoice ID or client name.
- Filter invoices by paid and pending status.
- Test the available sorting options.
- Mark a pending invoice as paid and verify the result.
- Delete an invoice and check that it is removed.
- Add a client, search for the client, edit the details and delete the record.
- Submit incomplete or invalid information and check the error messages.
- Check how the application behaves when the backend is unavailable.
- Test the interface on desktop and mobile screen sizes.



ER DIAGRAM

